REPORT ON OBJECT-ORIENTED PROGRAMMING (OOP)

Topic: Project Team and Personnel Management System

Source Code Repository (GitHub): *OOP_Labw_03*

SECTION A: ANALYSIS AND DESIGN REPORT

A.1 Class Design

- **System Classes:**
  - `Employee` : Base personnel entity.
  - `SoftwareEngineer` : Specialised technical personnel.
  - `ProjectTeam` : Project team aggregation and budget management.
- **Inter-Class Relationships:**
  - **Inheritance:**
    `SoftwareEngineer` inherits publicly from `Employee` , inheriting its identification structure, invariant constraints, and polymorphically overriding monthly cost calculation and data display.
  - **Aggregation:**
    `ProjectTeam` maintains an unowned reference list (`List<Employee>`). The lifecycle of personnel is independent of the project team. The `Leader` role is a designated position selected directly from this aggregated collection.

1. Class `Employee` (Base Personnel)

- **Responsibility:** Manages base identification and financial attributes; handles salary adjustments; provides an overridable interface for monthly cost calculation and information display.
- **Logical Attributes:**
  - `Id: string` (Read-only / `public get` , `private set`) - Unique employee identifier.
  - `FullName: string` (Read-Write / `public get` , `public set`) - Full name.
  - `BaseSalary: double` (Read-only / `public get` , `private set`) - Base remuneration.

- **Business Invariants:**
  - `Id` and `FullName` must not be empty or consist solely of whitespace.
  - `BaseSalary >= 0`.
- **Constructors:**
  - `Employee()`: Defaults to `Id = "UNKNOWN"`, `FullName = "Unnamed employee"`, `BaseSalary = 0`.
  - `Employee(string id, string fullName)`: Initialises identity; defaults to `BaseSalary = 0`.
  - `Employee(string id, string fullName, double baseSalary)`: Fully initialises all attributes with boundary validation.
- **Methods:**
  - `IncreaseSalary(double amount): void`: Increases base salary by a fixed monetary value ($amount > 0$).
  - `IncreaseSalary(double value, bool byPercentage): void`: Increases salary by a percentage rate if `true`, or by a fixed value if `false` ($value > 0$).
  - `virtual CalculateMonthlyCost(): double`: Computes monthly personnel cost (defaults to `BaseSalary`).
  - `virtual DisplayInfo(): void`: Renders personnel details to the standard output.
  - `~Employee()`: Finaliser logging the deallocation notice of the instance.

2. Class `SoftwareEngineer` (Software Engineer)

- **Responsibility:** Represents technical personnel; introduces primary programming language and technical allowance; overrides actual cost calculation and display behaviours.
- **Inheritance:** `Employee`.
- **Additional Attributes:**
  - `PrimaryLanguage: string` (Read-Write / `public get`, `public set`) - Core programming language.
  - `TechnicalAllowance: double` (Read-only / `public get`, `private set`) - Specialised technical allowance.
- **Business Invariants:**
  - Inherits all base invariants from `Employee`.
  - `PrimaryLanguage` must not be empty or whitespace.
  - `TechnicalAllowance >= 0`.
- **Constructors:**

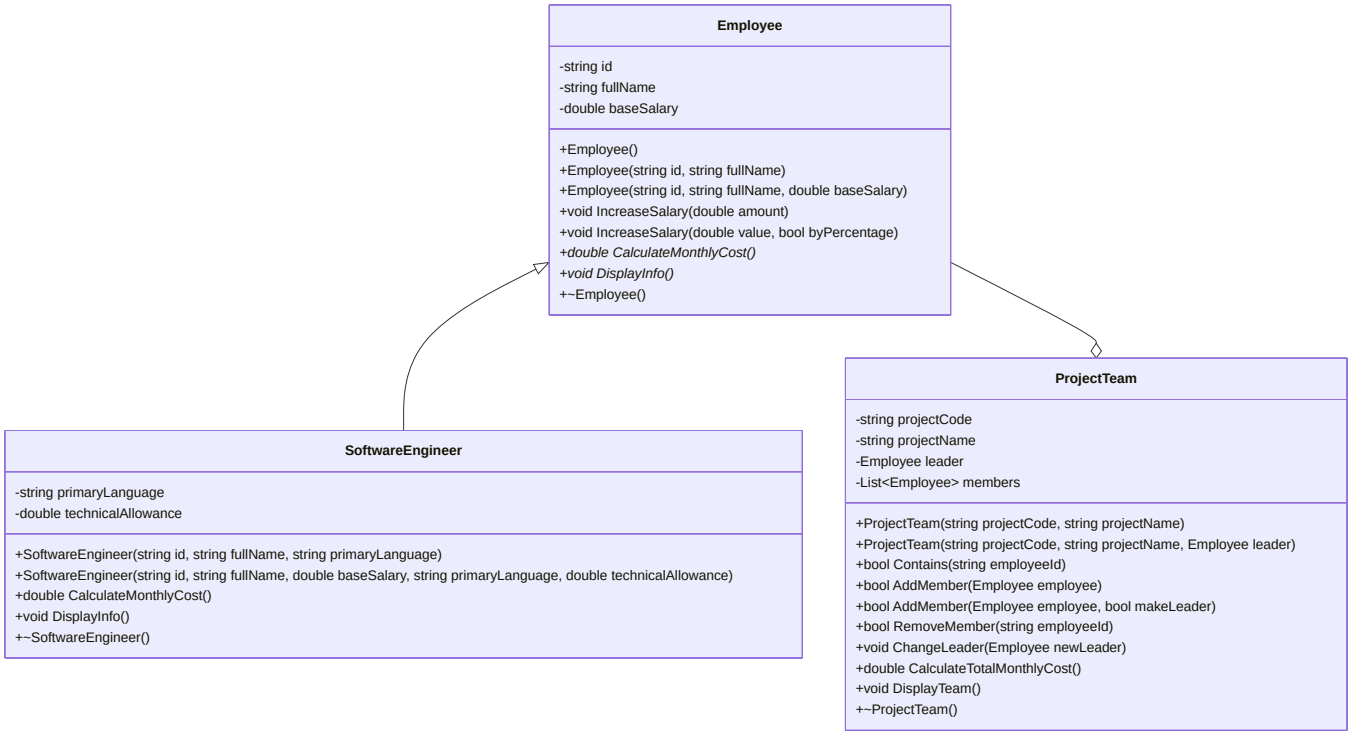
    - `SoftwareEngineer(string id, string fullName, string primaryLanguage)` : Initialises identity and language; defaults to `BaseSalary = 0`, `TechnicalAllowance = 0`.
    - `SoftwareEngineer(string id, string fullName, double baseSalary, string primaryLanguage, double technicalAllowance)` : Fully initialises base and derived attributes via constructor delegation.
- **Method Overrides:**
    - `override CalculateMonthlyCost(): double` : Returns `BaseSalary + TechnicalAllowance`.
    - `override DisplayInfo(): void` : Renders base attributes alongside primary language and technical allowance.
    - `~SoftwareEngineer()` : Finaliser logging the deallocation notice of the instance.

3. Class `ProjectTeam` (Project Team)

- **Responsibility:** Manages project team structure, maintains member rosters and leadership roles, and aggregates total monthly project remuneration.
- **Logical Attributes:**
    - `ProjectCode: string` (Read-only / `public get`, `private set`) - Unique project code.
    - `ProjectName: string` (Read-Write / `public get`, `public set`) - Project title.
    - `Leader: Employee?` (Read-only / `public get`, `private set`) - Reference to the designated Team Leader.
    - `Members: List<Employee>` (Read-only / `public get`, `private set`) - Unowned reference list of team members.
- **Business Invariants:**
    1. `ProjectCode` and `ProjectName` must not be empty or whitespace.
    2. *Uniqueness:* No two members in `Members` may share the same `Id`; an employee cannot appear more than once.
    3. *Leadership Integrity:* The team leader must strictly be a member of `Members`. Removing an active leader is prohibited unless a successor is appointed beforehand.
    4. *Lifecycle Independence:* Disbanding or deallocating a project team does not destroy external `Employee` instances.
- **Constructors:**
    - `ProjectTeam(string projectCode, string projectName)` : Initialises an empty team without an assigned leader (`Leader = null`).

    - `ProjectTeam(string projectCode, string projectName, Employee leader)`: Initialises team, assigns `Leader`, and automatically registers `leader` into `Members`.
- **Methods:**
    - `Contains(string employeeId): bool`: Determines whether an employee identifier is present within `Members`.
    - `AddMember(Employee employee): bool`: Enrols a member; returns `false` if `employeeId` already exists, otherwise appends and returns `true`.
    - `AddMember(Employee employee, bool makeLeader): bool`: Enrols a member; if `makeLeader == true`, concurrently assigns the entity as `Leader`.
    - `RemoveMember(string employeeId): bool`: Removes a member by ID. Returns `false` if the target is the active `Leader`; otherwise removes from list and returns `true`.
    - `ChangeLeader(Employee newLeader): void`: Reassigns leadership (automatically enrols `newLeader` into `Members` if not already present).
    - `CalculateTotalMonthlyCost(): double`: Aggregates the total monthly budget via polymorphic invocations of `CalculateMonthlyCost()` across all members.
    - `DisplayTeam(): void`: Outputs project overview, leader details, and the full member roster.
    - `~ProjectTeam()`: Finaliser clears internal references (`_members.Clear()`) without destroying referenced `Employee` objects.

A.2 UML Class Diagram



SECTION B: PROGRAMMING IMPLEMENTATION

The architecture is implemented in C# (.NET) across individual class files within the `Classes/` directory and verified via the `Test.cs` test harness:

1. **Encapsulation:**
   - State fields are strictly declared `private`.
   - Access is controlled through properties with backing fields, enforcing validation rules (rejecting blank strings and negative values).
2. **Overloading Techniques:**
   - Constructor overloading enables flexible instantiation ranging from minimal defaults to comprehensive parameters.
   - `IncreaseSalary` and `AddMember` are overloaded to support distinct business operational contexts.
3. **Polymorphism:**
   - `CalculateMonthlyCost()` and `DisplayInfo()` are declared `virtual` in base class `Employee` and overridden via `override` in `SoftwareEngineer`.
   - `ProjectTeam` iterates across the generic collection and triggers runtime dynamic dispatch without requiring explicit type casting.
4. **Lifecycle Management and Aggregation:**

- `ProjectTeam` maintains non-owning references to `Employee` instances.
- The finaliser of `ProjectTeam` clears internal collection references (`_members.Clear()`) without interfering with or deallocating the underlying `Employee` instances in memory.

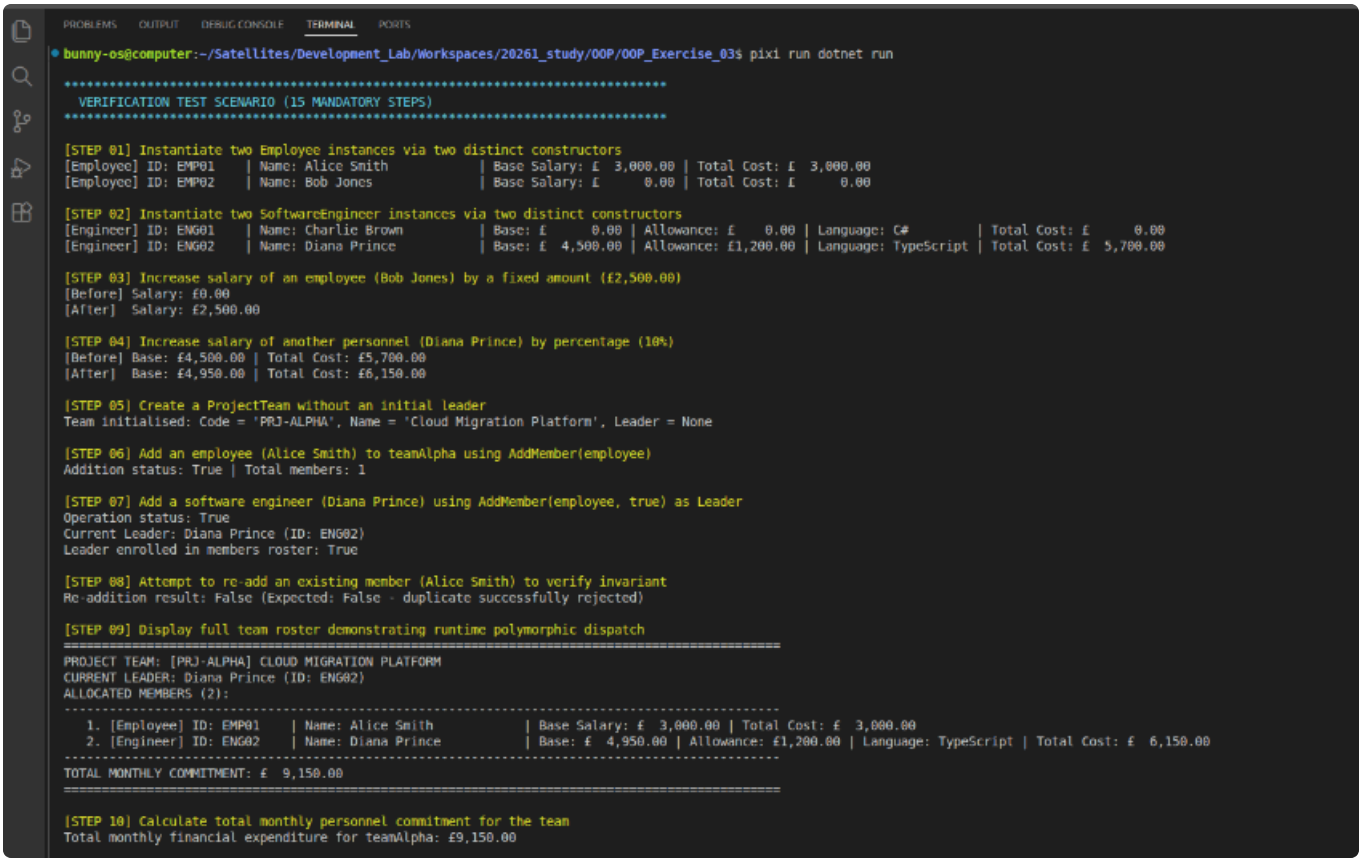
SECTION C: TEST SCENARIOS AND RESULTS

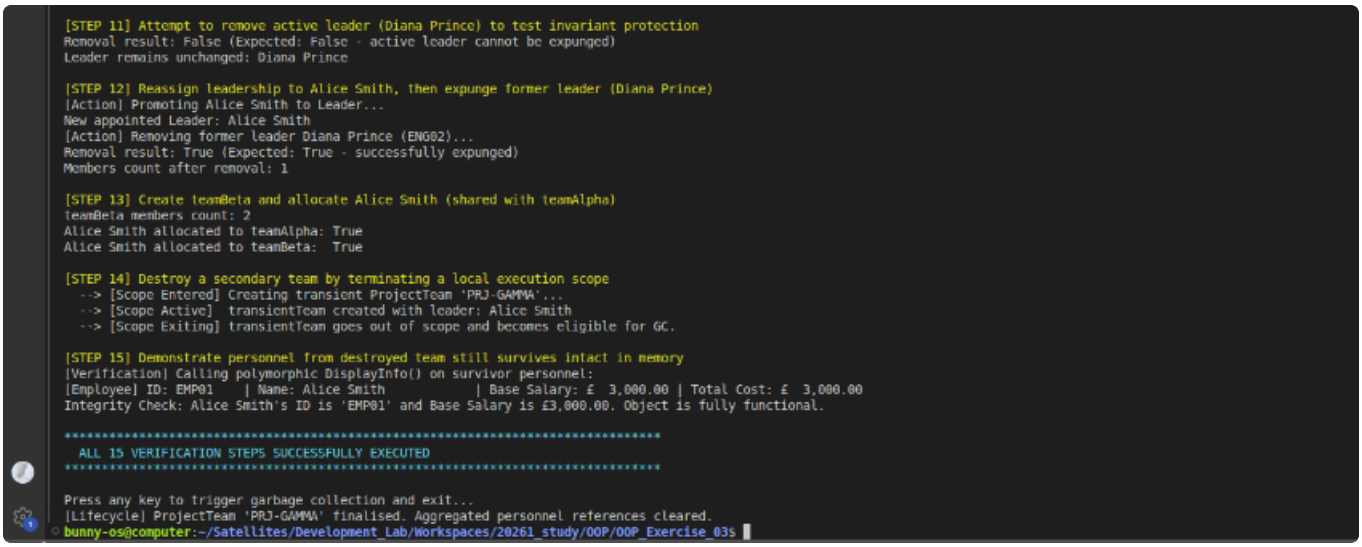
The automated verification harness in `Test.cs` sequentially executes **15 distinct test steps** conforming to the specification:

Step	Test Objective	Expected Outcome
01	Instantiate two `Employee` objects via two distinct constructors.	Successful initialisation with valid metadata and base salary.
02	Instantiate two `SoftwareEngineer` objects via two distinct constructors.	Successful initialisation with programming language and technical allowance.
03	Increase employee base salary by a fixed monetary amount.	Base salary increments accurately by the specified absolute value.
04	Increase another employee base salary by a percentage rate.	Base salary increments accurately according to the specified percentage.
05	Instantiate a project team without an assigned leader.	`Leader = null` ; internal member collection initialised empty.
06	Enrol an employee into the team via `AddMember(employee)` .	Returns `true` ; total member count increments to 1.
07	Enrol an engineer via `AddMember(employee, true)` as leader.	Returns `true` ; instance designated as `Leader` and registered in `Members` .
08	Attempt to enrol a duplicate employee identifier into the team.	Returns `false` (duplicate identifier rejected).
09	Render team roster via polymorphic invocation.	Polymorphic dispatch outputs categoric details specific to each concrete type.
10	Calculate total monthly project personnel cost.	Aggregated cost equals the exact sum of individual monthly costs.
11	Attempt to remove the active leader from the member collection.	Returns `false` (operation rejected to preserve leadership integrity).

Step	Test Objective	Expected Outcome
12	Appoint a new leader and subsequently remove the former leader.	Leadership reassignment succeeds; subsequent removal returns `true`.
13	Instantiate a second team and enrol an employee from the first team.	The identical employee reference is validly aggregated across both teams.
14	Deallocate the second team by exiting a local execution scope.	Second team goes out of scope and becomes eligible for garbage collection.
15	Verify employee from the second team persists post-team destruction.	Invoking methods on the employee instance confirms state remains fully intact.

- **Test Execution Results:**




[STEP 11] Attempt to remove active leader (Diana Prince) to test invariant protection
Removal result: False (Expected: False - active leader cannot be expunged)
Leader remains unchanged: Diana Prince
[STEP 12] Reassign leadership to Alice Smith, then expunge former leader (Diana Prince)
[Action] Promoting Alice Smith to Leader...
New appointed Leader: Alice Smith
[Action] Removing former leader Diana Prince (ENG02)...
Removal result: True (Expected: True - successfully expunged)
Members count after removal: 1
[STEP 13] Create teamBeta and allocate Alice Smith (shared with teamAlpha)
teamBeta members count: 2
Alice Smith allocated to teamAlpha: True
Alice Smith allocated to teamBeta: True
[STEP 14] Destroy a secondary team by terminating a local execution scope
--> [Scope Entered] Creating transient ProjectTeam 'PRJ-GAMMA'...
--> [Scope Active] transientTeam created with leader: Alice Smith
--> [Scope Exiting] transientTeam goes out of scope and becomes eligible for GC.
[STEP 15] Demonstrate personnel from destroyed team still survives intact in memory
[Verification] Calling polymorphic DisplayInfo() on survivor personnel:
[Employee] ID: EMP01 | Name: Alice Smith | Base Salary: £ 3,000.00 | Total Cost: £ 3,000.00
Integrity Check: Alice Smith's ID is 'EMP01' and Base Salary is £3,000.00. Object is fully functional.
****
ALL 15 VERIFICATION STEPS SUCCESSFULLY EXECUTED
****
Press any key to trigger garbage collection and exit...
[Lifecycle] ProjectTeam 'PRJ-GAMMA' finalised. Aggregated personnel references cleared.
bunny-os@computer:~/Satellites/Development_Lab/Workspaces/20261_study/OOP/OOP_Exercise_03s